

Lab 1: Self-Healing & Scaling

Kubernetes Fundamentals — Hour 1 Practical Assignment

Scenario

Your team lead asks you to prove that Kubernetes can keep a simple web application alive no matter what is thrown at it, and that it can run multiple copies of that application simultaneously. Before connecting a database or an API layer, the team wants confidence in the underlying self-healing and scaling behavior.

Learning Objectives

- Deploy a multi-replica application using a Deployment manifest
- Observe how Kubernetes responds when a Pod is deleted
- Scale a running Deployment imperatively, without editing YAML
- Read and interpret kubectl describe output for a Deployment

Prerequisites

- Docker Desktop with Kubernetes enabled (kind or Kubeadm, single node)
- kubectl confirmed working: kubectl get nodes shows a Ready node

Instructions

Step 1 — Write the manifest

Create a file named `lab1-nginx.yaml` containing a Deployment and a Service. The Deployment must run the `nginx` image with 3 replicas. The Service must expose port 80 and route to the Deployment's Pods.

Think before you write

What label will the Service use to find the right Pods?
Where does replica count belong in the manifest?

Step 2 — Deploy and verify

1. Apply the manifest to the cluster.
2. Confirm all 3 Pods reach the Running state.

3. Confirm the Service was created with a ClusterIP.

Step 3 — Break it on purpose

4. List the running Pods and note their exact names.
5. Delete exactly one of the three Pods.
6. Immediately list the Pods again.
7. Record what changed: pod name, age, and total count.

Step 4 — Scale without touching the YAML file

8. Find the kubectl command that changes replica count directly from the command line, without re-applying any file.
9. Use it to scale the Deployment from 3 replicas to 5.
10. List the Pods again and confirm 5 are now running.

Step 5 — Read the Deployment's own report

11. Run the describe command against the Deployment (not a Pod).
12. Locate the field that shows current replica count versus desired replica count.
13. Locate the Events section and note the most recent scaling event.

Your Answers

Fill in what you actually ran and observed. One or two sentences per item is enough.

Command used to deploy the manifest

```
kubectl apply -f lab1-nginx.yaml
```

```
[vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 43 June 24 % kubectl apply -f lab1-nginx.yaml
deployment.apps/nginx-demo unchanged
service/nginx-demo unchanged
```

What happened immediately after deleting one Pod, and why

Self Healing concept used, pod counts remain same, one deleted and new pod generated

```
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 43 June 24 % kubectl delete pod nginx-demo-dd58d7d-bs2k4
pod "nginx-demo-dd58d7d-bs2k4" deleted from default namespace
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 43 June 24 % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-demo-dd58d7d-kdpvq            1/1     Running   0           6s
nginx-demo-dd58d7d-rp227            1/1     Running   0          6m40s
nginx-demo-dd58d7d-xmzb7            1/1     Running   0          6m40s
postgres-54b955fd9b-f2dts          1/1     Running   0          13m
```

Command used to scale from 3 to 5 replicas

kubectl scale deployment nginx-demo --replicas=5

```
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 43 June 24 % kubectl scale deployment nginx-demo --replicas=5
deployment.apps/nginx-demo scaled
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 43 June 24 % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-demo-dd58d7d-kdpvq            1/1     Running   0           62s
nginx-demo-dd58d7d-krxsr            1/1     Running   0           4s
nginx-demo-dd58d7d-rp227            1/1     Running   0          7m36s
nginx-demo-dd58d7d-smwkr            1/1     Running   0           4s
nginx-demo-dd58d7d-xmzb7            1/1     Running   0          7m36s
postgres-54b955fd9b-f2dts          1/1     Running   0          14m
```

Where current vs. desired replica count appears in describe output

In Replicas

```
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 43 June 24 % kubectl describe deployment nginx-demo
Name:                               nginx-demo
Namespace:                           default
CreationTimestamp:                   Wed, 24 Jun 2026 12:28:07 +0530
Labels:                               app=nginx-demo
Annotations:                         deployment.kubernetes.io/revision: 1
Selector:                             app=nginx-demo
Replicas:                             5 desired | 5 updated | 5 total | 5 available | 0 unavailable
StrategyType:                         RollingUpdate
MinReadySeconds:                       0
RollingUpdateStrategy:                 25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx-demo
  Containers:
    nginx:
      Image:          nginx:latest
      Port:            80/TCP
      Host Port:       0/TCP
      Environment:    <none>
      Mounts:          <none>
      Volumes:         <none>
      Node-Selectors:  <none>
      Tolerations:     <none>
Conditions:
  Type           Status  Reason
  ----           -
  Progressing    True    NewReplicaSetAvailable
  Available      True    MinimumReplicasAvailable
OldReplicaSets:  <none>
NewReplicaSet:   nginx-demo-dd58d7d (5/5 replicas created)
Events:
  Type           Reason             Age   From               Message
  ----           -
  Normal         ScalingReplicaSet   8m3s  deployment-controller  Scaled up replica set nginx-demo-dd58d7d from 0 to 3
  Normal         ScalingReplicaSet   31s   deployment-controller  Scaled up replica set nginx-demo-dd58d7d from 3 to 5
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 43 June 24 %
```

Solution

Before you read on

Only continue once you've attempted Steps 1–5 yourself, or are genuinely stuck.

Step 1 — The manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-demo
  labels:
    app: nginx-demo
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-demo
  template:
    metadata:
      labels:
        app: nginx-demo
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-demo
spec:
  selector:
    app: nginx-demo
  ports:
    - port: 80
      targetPort: 80
```

Step 2 — Deploy and verify

```
kubectl apply -f lab1-nginx.yaml
kubectl get pods
kubectl get deployments
kubectl get services
```

Expected: 3 Pods named nginx-demo-`<hash>-<id>`, all eventually showing 1/1 Running. The Deployment shows READY 3/3. The Service shows TYPE ClusterIP with port 80.

Step 3 — Delete a Pod

```
kubectl get pods
kubectl delete pod nginx-demo-<hash>-<id>
kubectl get pods
```

Expected observation: the deleted Pod disappears, and within seconds a brand new Pod appears with a different random suffix and an age of just a few seconds. The total count returns to 3 almost immediately.

Why this happens: the Deployment controller continuously compares desired state (3 replicas, as declared in the manifest) against actual state (how many matching Pods currently exist). Deleting a Pod drops the actual count to 2, which no longer matches the desired count of 3. The controller's reconciliation loop detects this mismatch and creates a replacement Pod automatically. This is Kubernetes' self-healing behavior: you declare an end state, and the system continuously works to maintain it, regardless of what disrupts it.

Step 4 — Scale imperatively

```
kubectl scale deployment nginx-demo --replicas=5
kubectl get pods
```

Expected observation: 2 additional Pods appear within seconds, bringing the total to 5, all eventually Running. The `kubectl scale` command edits the Deployment's desired replica count directly, on the live cluster object, without touching the original YAML file on disk. The YAML file and the cluster's actual state can diverge this way — worth remembering, since re-applying the original file later would reset replicas back down to 3.

Step 5 — Reading describe output

```
kubectl describe deployment nginx-demo
```

Look for a line near the top of the output that reads something like Replicas: 5 desired | 5 updated | 5 total | 5 available | 0 unavailable. This single line reports desired vs. actual state side by side — the exact comparison the reconciliation loop is always making internally.

The Events section at the bottom lists a ScalingReplicaSet event with a message such as Scaled up replica set nginx-demo-`<hash>` to 5, timestamped to match when the scale command was run.

Key takeaway

Everything in this lab traces back to one idea: a Deployment does not run Pods directly — it continuously enforces a declared replica count. Deleting a Pod, scaling up, and recovering from a crash are all the same mechanism responding to different triggers.